



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования

"МИРЭА - Российский технологический университет"

РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра математического обеспечения и стандартизации информационных
технологий (МОСИТ)

ОТЧЁТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №1
по дисциплине
«Структуры и алгоритмы обработки данных»

Тема: вычислительная сложность алгоритма

Выполнил студент группы ИКБО-64-23

Астахов С.П.

Принял старший преподаватель кафедры

Скворцова Л.А.

Москва 2023

Содержание

1. ПОСТАНОВКА ЗАДАЧИ И УСЛОВИЕ.....	3
2. РАЗРАБОТКА РЕШЕНИЯ	5
2.1 Таблица псевдокода	5
2.2 Функциональные зависимости	6
2.3 Код алгоритма на C++.....	6
2.4 Тестирование алгоритма.....	7
2.5 Скрины результатов тестирования.....	8
2.6 Исследование алгоритма	9
3 ВЫВОДЫ.....	12
4 СПИСОК ИНФОРМАЦИОННЫХ ИСТОЧНИКОВ	13

1. ПОСТАНОВКА ЗАДАЧИ И УСЛОВИЕ

Задание: разработать алгоритм задачи варианта. Определить функцию, показывающую зависимость количества выполняемых инструкций от размера задачи, функцию емкостной сложности алгоритма.

Требования к выполнению работы:

1. Выполнить разработку алгоритма задачи варианта, представляя последовательность как массив из n значений, записать алгоритм на псевдокоде.
2. Определить, для полученного алгоритма, функциональную зависимость (функцию), указывающей зависимость количества выполняемых инструкций от размера задачи.
3. Технологию подсчета количества инструкций алгоритма представить в таблице табл. 1. Псевдокод алгоритма разместить в столбце Инструкции (оператор) алгоритма таблицы (табл.2), где каждая управляющая инструкция строго в отдельной строке таблицы.
4. Определить функцию (наилучший, наихудший, средний случай) зависимости количества инструкций алгоритма от размера задачи и от данных
5. Реализовать алгоритм
6. Разработать тесты для доказательства корректной работы алгоритма при $n=20$.
Подготовку тестов выполнить в таблице (табл. 3)
7. Выполнить отладку и тестирование алгоритма.
8. Определить емкостную сложность алгоритма

Индивидуальный вариант (№6):

Условие: Дано натуральное число n и последовательность действительных чисел $a_1, \dots, a_n$. Определить количество натуральных чисел j ($2 \leq j \leq n-1$), для которых $a_{j-1} < a_j > a_{j+1}$.

Предусловие: $a[]$ - массив целых значений длины n ($n > 0$)

Постусловие: Количество натуральных чисел j ($2 \leq j \leq n-1$)

Таблица определения имен объектов алгоритма:

Таблица 1. Объекты алгоритма

Имя	Диапазон допустимых значений	Ограничения	Семантика (что определяет имя)
n	$(1; +\infty)$	$n > 0$	int
c	$(-\infty; +\infty)$	-	float
sum	$(0; +\infty)$	$\text{sum} \geq 0$	int
a	$(-\infty; +\infty)$	$\text{len}(a) = n$	Vector (изм. массив)

2. РАЗРАБОТКА РЕШЕНИЯ

2.1 Таблица псевдокода

Псевдокод - компактный, зачастую неформальный язык описания алгоритмов, использующий ключевые слова императивных языков программирования, но опускающий несущественные для понимания алгоритма подробности и специфический синтаксис.

Таблица 2. Форма представления алгоритма при получении функции зависимости количества выполняемых инструкций от размера задачи

Номер строки инструкции алгоритма	Инструкция (оператор) алгоритма	Количество выполненных операций на инструкцию
1	$\text{sum} \leftarrow 0$	1
2	for $j \leftarrow 1$ to $(n-1)$ do	$3 \cdot (n+1-1) = 3n$
3	if $(a[j-1] < a[j] \text{ and } a[j] > a[j+1])$ then	$8 \cdot (n-1)$
4	$\text{sum} \leftarrow \text{sum} + 1$	$2 \cdot (n-1)$
5	od	

Рассмотрим три случая (на всех наборах значений массива $a[]$, худший, лучший, средний) выполнения алгоритма и оценим его сложность с помощью функции $f(n)$, которая представляет из себя сумму всех выполненных на инструкцию

операций. Полученное выражение будет называться функциональной зависимостью алгоритма.

2.2 Функциональные зависимости

$$f_l(n) = 1 + 3n + 8(n - 1) = 1 + 3n + 8n - 8 = 11n - 7 \quad (1)$$

Формула (1) показывает теоретическую сложность алгоритма, если все элементы массива равны (лучший случай), представляет из себя выражение $11n - 7$. Рассмотрим следующий случай алгоритма:

$$f_x(n) = 1 + 3n + 8 * (n - 1) + 2 * (n - 1)/2 = 12n - 8 \quad (2)$$

Формула (2) показывает теоретическую сложность алгоритма если все значения массива различные и предыдущий и следующие значения массива меньше, чем нынешний (их будет всего половина). Также составим формулу для среднего значения (3)

$$f_c(n) = \frac{((11n - 7) + (12n - 8))}{2} = (23n - 15)/2 \quad (3)$$

Емкостная сложность алгоритма: на вход поступает массив размера n , дополнительных структур данных не требуется.

2.3 Код алгоритма на C++

Запишем код алгоритма на C++, который будет выполнять инструкции в соответствие с записанным ранее псевдокодом:

```

int number_of_j(int n, const vector<float> &a) {
    int sum = 0;
    for (int j = 1; j <= (n-1); j++) { // в задаче подразумевается что отчёт j начинается с 2,
        //но т.к. массив начинается с нулевого элемента, j=1
        if (a[j-1] < a[j] && a[j] > a[j+1]) {
            sum += 1;
        }
    }
    return sum;
}

```

Рисунок 1. Код алгоритма на языке C++

2.4 Тестирование алгоритма

Разработаем тесты при $n = 10$, с которыми мы будем сравнивать результаты работы алгоритма, записанного выше.

Таблица 3. Таблица тестов алгоритма

Номер теста	Входные данные	Эталон результата (ожидаемый результат)
1	$n = 10$ $a\{0, 2, 0, 2, 0, 2, 2, 0, 2, 0\}$	3
2	$n = 10$ $a\{-1, 3.2, -2, 3, 5, -2.99, 7, 6, 6, 0\}$	3
3	$n = 10$ $a\{1, 7, 3, 2, 5, 1, 7, 1, 7, 3\}$	4

Проведя подсчёты вручную, мы получили ожидаемые от программы результаты вычисления. Теперь нам необходимо прогнать эти же значения на написанном алгоритме и убедиться в корректности его работы.

2.5 Скрины результатов тестирования

Прогнав значения из таблицы (3) через алгоритм написанный на C++, мы получили все те же самые результаты, что говорит нам о корректности написанного алгоритма. Для удобства добавим вводные значения для n и элементов массива a (рис.2):

```
int main() {
    setlocale(0, "Rus");
    int n; float c;
    cout << "Введите количество элементов в массиве: "; cin >> n;
    vector<float> a;

    for (int i = 1; i <= n; i++){
        cout << i << ") "; cin >> c;
        a.push_back(c);
    }

    cout << "Кол-во чисел, для которых  $a(j-1) < a_j > a(j+1)$ : " << number_of_j(n, a);
}
```

Рисунок 2. Алгоритм для записи длины и чисел в массиве

Предоставим скрины тестирования в среде разработки:

```
Введите количество элементов в массиве: 10
1) 0
2) 2
3) 0
4) 2
5) 0
6) 2
7) 2
8) 0
9) 2
10) 0
Кол-во чисел, для которых  $a(j-1) < a_j > a(j+1)$ : 3
```

Рисунок 3. Вывод результатов работы 1-го теста в консоли


```
Введите количество элементов в массиве: 10
1) -1
2) 3.2
3) -2
4) 3
5) 5
6) -2.99
7) 7
8) 6
9) 6
10) 0
Кол-во чисел, для которых  $a(j-1) < a_j > a(j+1)$ : 3
```

Рисунок 4. Вывод результатов работы 2-го теста в консоли

```
Введите количество элементов в массиве: 10
1) 1
2) 7
3) 3
4) 2
5) 5
6) 1
7) 7
8) 1
9) 7
10) 3
Кол-во чисел, для которых  $a(j-1) < a_j > a(j+1)$ : 4
```

Рисунок 5. Вывод результатов работы 3-го теста в консоли

2.6 Исследование алгоритма

Теперь нам необходимо измерить временную сложность написанного алгоритма, просчитать количество инструкций $[f(n)]$ при разных значениях n и найти время выполнения этих инструкций. Результаты занести в таблицу.

Для измерения временной сложности алгоритма используем модуль `chrono` из стандартной библиотеки языка C++.

Предоставленные значения n при которых необходимо проводить тестирование: 1, 100, 1000, 5000.

```

int number_of_j(int n, const vector<float> &a) {

    auto beging_t = chrono::steady_clock::now();

    int sum = 0;
    for (int j = 1; j <= (n-1); j++) { // в задаче подразумевается что отчёт j начинается с 2,
        //но т.к. массив начинается с нулевого элемента, j=1
        if (a[j..- 1] < a[j] && a[j] > a[j..+ 1]) {
            sum += 1;
        }
    }

    auto end_t = chrono::steady_clock::now();
    chrono::duration<double, std::nano> elapsed_time = end_t - beging_t;
    cout << elapsed_time.count() << '\n';

    return sum;
}

```

Рисунок 5. Алгоритм замера времени от начала алгоритма до конца

Данный алгоритм предназначен замера времени в наносекундах от инструкции инициализации sum до инициализации оператора cout, что позволяет выполнить замер максимально точно. Сделаем прогон на разных массивах размера n и запишем результат.

На практике вышло так, что каждый прогон одного и того же значения даёт разный результат, поэтому имеет смысл найти среднее из результатов нескольких замеров и внести их в таблицу.

При измерении времени при $n = 1$ мы получили следующие значения: 100 200 200 300 200. Их среднее арифметическое будет 200 нс. В таблицу требуется занести временные данные в секундах. Для перевода следует разделить полученное значение на 1 000 000 000.

$$\frac{200[нс]}{10^9[нс]} = 0.0000002[сек] \quad (4)$$

Проделаем все те же действия и измерения при других n .

Быстродействие компьютера характеризуется тактовой частотой процессора (в ГГц), которая выражает количество операций/сек совершаемых процессором.

Пускай процессор имеет тактовую частоту 3.5 ГГц, в таком случае он сможет выполнить 3.5×10^9 операций за секунду (Гц).

Таблица 4. Параметры алгоритма при оценке сложности алгоритма

Размер задачи (n)	Время выполнения алгоритма (сек)	Количество операций по полученной ранее формуле	Время выполнения Т инструкций на компьютере (Т / f) (сек)
1	0.0000002	4	0.000000000114
100	0.0000012	1 1423	0.0000000369
1000	0.0000097	11 493	0.000000371
5000	0.00005	57 493	0.00000186

3 ВЫВОДЫ

В ходе данной практической работы мы научились находить формулу зависимости количества операций алгоритма от длины массива n . Увидели линейный рост времени в зависимости от значения переменной n , что показано в таблице 4. Также мы научились измерять время выполнения алгоритма с помощью модуля `chrono` и время выполнения $f(n)$ операций на компьютере и определять емкостную сложность алгоритма.

4 СПИСОК ИНФОРМАЦИОННЫХ ИСТОЧНИКОВ

1. Скворцова Л.А, Гусев К.В, Трушин С.М, Филатов А.С: Учебно-методическое пособие СИАОД часть 1 / — МИРЭА — Российский-технологический университет, 2021. – 146 с
2. Лекционный материал Скворцовой Л.А (старший преподаватель кафедры МОСИТ)